# TEST CASE LIBRARY

Web Application — Azure DevOps Test Suite

Brian Scott McCarthy | Senior QA Lead | briansmc@gmail.com

## Module 1: User Authentication

### TC-0001 [P1] User Registration — Valid New Account

*Suite: TS-Authentication-Functional | Preconditions: App is accessible; no existing account with test email.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Navigate to /register and verify registration form renders with fields: First Name, Last Name, Email, Password, Confirm Password. | Registration form displays all required fields with correct labels and placeholder text. |
| 2 | Enter valid data: First Name='Test', Last Name='User', Email='testuser@qa.com', Password='SecureP@ss1!', Confirm Password='SecureP@ss1!'. | All fields accept input without error. |
| 3 | Click 'Create Account' button. | Form submits; loading indicator appears briefly. |
| 4 | Observe post-submission behaviour. | User is redirected to /dashboard; welcome message displays user's first name. |
| 5 | Validate via SQL: SELECT * FROM users WHERE email='testuser@qa.com' | Record exists in DB with correct name, email, hashed password (not plain text), created_at timestamp. |

**Overall Expected: Account is created successfully; user is authenticated and redirected to dashboard; DB record confirmed.**

### TC-0002 [P1] User Login — Valid Credentials

*Suite: TS-Authentication-Functional | Preconditions: Existing account with email='testuser@qa.com', password='SecureP@ss1!'.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Navigate to /login. | Login page renders with Email and Password fields and 'Sign In' button. |
| 2 | Enter email='testuser@qa.com' and password='SecureP@ss1!'. | Fields accept input. |
| 3 | Click 'Sign In'. | Form submits; loading indicator shown. |
| 4 | Observe post-login state. | User is redirected to /dashboard; session cookie/JWT token is set; nav bar shows authenticated user name. |

**Overall Expected: User authenticates successfully and accesses dashboard.**

### TC-0003 [P2] User Login — Invalid Password (Negative)

*Suite: TS-Authentication-Negative | Preconditions: Existing account for testuser@qa.com.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Navigate to /login and enter email='testuser@qa.com', password='WrongPass123!'. | Fields accept input without immediate error. |

| # | Test Step | Expected Result |
|---|---|---|
| 2 | Click 'Sign In'. | Form submits. |
| 3 | Observe response. | Error message displays: 'Invalid email or password.' User remains on /login page. No account lockout on first attempt. |
| 4 | Repeat incorrect login 4 more times (5 total). | After 5th failed attempt, account lockout message is displayed and user cannot attempt login for 15 minutes. |

**Overall Expected: Invalid credentials are rejected gracefully; lockout policy is enforced after 5 failures.**

### TC-0004 [P2] Password Reset — Email Flow

*Suite: TS-Authentication-Functional | Preconditions: Account exists for testuser@qa.com.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Click 'Forgot Password?' on /login page. | Password reset page renders with Email input. |
| 2 | Enter 'testuser@qa.com' and click 'Send Reset Link'. | Success message: 'If this email is registered, a reset link has been sent.' |
| 3 | Check test email inbox (Mailinator or QA inbox). | Email received within 2 minutes with a reset link. |
| 4 | Click reset link. | Redirected to /reset-password with token param; new password form displayed. |
| 5 | Enter and confirm new password 'NewSecure@99!' and submit. | Success message shown; user redirected to /login. |
| 6 | Log in with new password. | Login succeeds. |

**Overall Expected: Password reset completes successfully via email flow; old password is invalidated.**

# Module 2: Product Search & Browse

### TC-0010 [P1] Keyword Search — Valid Results

*Suite: TS-Search-Functional | Preconditions: User is logged in; product catalogue contains at least 5 items with keyword 'sofa'.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Click the search bar in the navigation header. | Search input is focused and accepts keyboard input. |
| 2 | Type 'sofa' and press Enter. | Search executes; URL updates to /search?q=sofa. |
| 3 | Verify search results page. | Results grid renders with product cards containing image, name, price, and 'Add to Cart' CTA; result count displayed (e.g. '24 results for sofa'). |
| 4 | Validate result relevance — all displayed products contain 'sofa' in name or description. | No irrelevant products displayed in first page of results. |
| 5 | Check pagination if >12 results. | Pagination controls visible; clicking page 2 loads next set without page reload. |

**Overall Expected: Keyword search returns relevant, paginated results with correct product data displayed.**

### TC-0011 [P2] Search — Filter by Price Range

*Suite: TS-Search-Functional | Preconditions: On /search results page with results loaded.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Locate the 'Price' filter in the sidebar. | Price range slider or min/max input fields displayed. |
| 2 | Set price range to $100 – $500 and apply filter. | Results reload; URL updates with price filter params. |
| 3 | Verify all displayed products are priced between $100 and $500. | No product outside the specified range is visible on the page. |
| 4 | Clear the filter. | All results restore to pre-filter state. |

**Overall Expected: Price filter correctly constrains results to the specified range; filter can be cleared.**

### TC-0012 [P2] Search — No Results (Edge Case)

*Suite: TS-Search-Negative | Preconditions: User is on search page.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Search for a string guaranteed to return no results: 'xyznonexistent123'. | Search executes. |
| 2 | Observe results area. | 'No results found' message displayed with suggestion to refine search; no broken layout or JS errors in console. |

**Overall Expected: Zero-result state handled gracefully with informative UI message.**

# Module 3: Shopping Cart & Checkout

### TC-0020 [P1] Add Item to Cart

*Suite: TS-Cart-Functional | Preconditions: User is logged in; product 'Product-SKU-001' is in stock.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Navigate to a product detail page. | Product page loads with correct name, image, price, and stock status. |
| 2 | Select quantity '2' from the quantity picker. | Quantity updates to 2. |
| 3 | Click 'Add to Cart'. | Cart icon in nav updates count to 2; success toast appears: 'Added to Cart'. |
| 4 | Open the cart panel/page. | Cart displays Product-SKU-001, quantity 2, correct line-item total (unit price × 2). |
| 5 | Validate via SQL: SELECT * FROM cart_items WHERE session_id='[session]' | Cart record exists with correct product_id, quantity, and price snapshot. |

**Overall Expected: Item added to cart with correct quantity and price; cart state persisted in DB.**

### TC-0021 [P1] Checkout — Complete Order with Credit Card

*Suite: TS-Checkout-Functional | Preconditions: Cart contains at least 1 item; user has a saved address; test Stripe card: 4242 4242 4242 4242.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Proceed from cart to /checkout. | Checkout page renders: Order Summary, Shipping Address, Payment sections. |
| 2 | Select saved shipping address. | Address populates; shipping cost calculated and displayed. |
| 3 | Enter test card details: 4242 4242 4242 4242, exp 12/26, CVV 123. | Payment fields accept input; card type icon shown (Visa). |
| 4 | Click 'Place Order'. | Order processing indicator shown; Stripe API call initiated. |
| 5 | Observe confirmation. | Order confirmation page shown with Order ID; confirmation email sent to user. |
| 6 | Validate via SQL: SELECT * FROM orders WHERE user_id='[id]' ORDER BY created_at DESC LIMIT 1 | Order record exists with correct total, status='confirmed', payment_status='paid'. |

**Overall Expected: Order placed successfully end-to-end; confirmation displayed; DB record accurate; email sent.**

# Module 4: API Testing — REST Endpoints

### TC-0030 [P1] GET /api/products — Returns Product List

*Suite: TS-API-Products | Preconditions: API environment is running; auth token available.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Open Postman collection 'Products API'. | Collection loads with pre-configured base URL and auth header. |
| 2 | Send GET /api/products with valid Bearer token. | Request executes. |
| 3 | Verify HTTP status code. | Response: 200 OK. |
| 4 | Verify response schema — array of product objects each containing: id, name, price, stock, imageUrl. | All fields present; correct data types (price is number, not string). |
| 5 | Verify response time. | Response received in under 800ms. |

**Overall Expected: API returns 200 with correctly structured product array within SLA.**

### TC-0031 [P1] POST /api/orders — Create New Order

*Suite: TS-API-Orders | Preconditions: Valid auth token; product with id='prod-001' in stock.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | In Postman, send POST /api/orders with body: {productId:'prod-001', quantity:1, shippingAddressId:'addr-001'}. | Request executes. |
| 2 | Verify HTTP status code. | 201 Created. |
| 3 | Verify response body contains: orderId, status:'pending', total, estimatedDelivery. | All fields present with valid values. |
| 4 | Send GET /api/orders/{orderId} to retrieve the created order. | 200 OK; order details match the POST payload. |

**Overall Expected: Order creation API returns 201 with valid order object; order retrievable via GET.**

### TC-0032 [P2] POST /api/orders — Unauthorised (Negative)

*Suite: TS-API-Security | Preconditions: No auth token or expired token.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Send POST /api/orders without Authorization header. | Request executes. |
| 2 | Verify HTTP status code. | 401 Unauthorized. |
| 3 | Verify response body. | Error message: {error:'Unauthorized', message:'Authentication required.'} — no sensitive data exposed. |

**Overall Expected: Unauthenticated API calls rejected with 401; no data leakage.**

# Module 5: Automation — Playwright Test Cases

### TC-0040 [P1] Playwright — End-to-End Login & Dashboard Load

*Suite: TS-Automation-Playwright | Preconditions: Playwright suite installed; .env configured with TEST_USER credentials; app deployed to QA.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Run: npx playwright test tests/auth/login.spec.ts | Test execution begins. |
| 2 | Script navigates to /login, fills credentials, clicks Sign In. | Browser automation executes without errors. |
| 3 | Script asserts: URL is /dashboard; page title contains 'Dashboard'; user greeting visible. | All assertions pass; Playwright report shows PASSED. |
| 4 | Assert no console errors during navigation. | Browser console log is clean (no uncaught exceptions). |

**Overall Expected: Playwright E2E login test passes; dashboard loads cleanly with no console errors.**

### TC-0041 [P1] Playwright — Cross-Browser Checkout Flow

*Suite: TS-Automation-CrossBrowser | Preconditions: Playwright configured for Chromium, Firefox, WebKit; test data seeded.*

| # | Test Step | Expected Result |
|---|---|---|
| 1 | Run: npx playwright test tests/checkout/checkout.spec.ts --project=chromium | Passes on Chromium. |
| 2 | Run same test with --project=firefox | Passes on Firefox. |
| 3 | Run same test with --project=webkit | Passes on WebKit (Safari). |
| 4 | Review Playwright HTML report. | All 3 browser projects show PASSED; screenshots attached for each. |

**Overall Expected: Checkout flow passes consistently across all 3 browser engines.**